From Bits to Pixels

The Mathematical and Physical Foundations of elementary 3D Graphics

Jack Wetherell

Contents

1 Foundations: Numbers, Memory, and Representation **2**
1.1 Bits, Bytes, and Binary Storage 2
1.2 Floating Point Representation (IEEE 754) 2
1.3 From Raw Memory to Numerical Values 3

2 Geometry Representation in Memory **4**
2.1 Vertices (Position Vectors) 4
2.2 Triangles (Faces) 4
2.3 Indexed vs Non-Indexed Geometry 5
2.4 Meshes (Objects) 5
2.5 Scenes as Collections of Objects 6

1 Foundations: Numbers, Memory, and Representation

1.1 Bits, Bytes, and Binary Storage

At the lowest level, all data in a computer is represented using binary digits.

- A **bit** is a single binary value:
$$b \in \{0, 1\} \tag{1}$$
- A **byte** consists of 8 bits:
$$1 \text{ byte} = 8 \text{ bits} \tag{2}$$

A sequence of bits can represent integers using positional notation in base 2:

$$N = \sum_{i=0}^{n-1} b_i \cdot 2^i \tag{3}$$

where:

- b_i is the i-th bit (0 or 1)
- $i = 0$ corresponds to the least significant bit (LSB)

Example:
$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10} \tag{4}$$

Memory is therefore a linear array of bytes:

$$\text{Memory} = [\text{byte}_0, \text{byte}_1, \ldots, \text{byte}_N] \tag{5}$$

Each byte has an address:

$$\text{Address} \in \mathbb{N} \tag{6}$$

1.2 Floating Point Representation (IEEE 754)

Real numbers are represented approximately using the IEEE 754 floating point format.

A 32-bit floating point number (single precision) is composed of:

- 1 bit for the sign (s)
- 8 bits for the exponent (e)
- 23 bits for the mantissa (fraction) (m)

The value represented is:

$$x = (-1)^s \cdot (1 + m) \cdot 2^{e-127} \tag{7}$$

where:

$$m = \sum_{i=1}^{23} m_i \cdot 2^{-i} \tag{8}$$

Interpretation:

- $(-1)^s$ determines the sign
- $(1 + m)$ is the normalized significand
- $e - 127$ is the unbiased exponent

Special Cases:

- $e = 0 \rightarrow$ denormalized numbers
- $e = 255 \rightarrow$ infinity or NaN

1.3 From Raw Memory to Numerical Values

All numerical values in a program are ultimately stored as sequences of bytes in memory.

A 32-bit float occupies 4 consecutive bytes:

$$\text{float} \rightarrow [b_0, b_1, b_2, b_3] \tag{9}$$

These bytes are interpreted according to the IEEE 754 structure:

$$\underbrace{s}_{1\text{ bit}} \quad \underbrace{e}_{8\text{ bits}} \quad \underbrace{m}_{23\text{ bits}} \tag{10}$$

Example: Representing $x = 7.98$ The decimal number 7.98 cannot be represented exactly in binary, so it is approximated.

Its IEEE 754 single-precision representation is:

$$x \approx 7.980000019073486 \tag{11}$$

Binary layout:

$$0\ 10000001\ 11111111001100110011010 \tag{12}$$

Breaking this down:

- Sign bit:
$$s = 0 \quad \text{(positive)} \tag{13}$$
- Exponent:
$$e = 10000001_2 = 129_{10} \tag{14}$$
$$e_{\text{unbiased}} = 129 - 127 = 2 \tag{15}$$
- Mantissa:
$$m = 1.11111111001100110011010_2 \tag{16}$$

Reconstructing the value:

$$x = (-1)^0 \cdot (1 + m_{\text{fraction}}) \cdot 2^2 \tag{17}$$

Key Insight: There is no intrinsic meaning in memory—only interpretation.

$$\text{Bits} \xrightarrow{\text{interpretation}} \text{Number} \tag{18}$$

The same sequence of bits may represent:

- an integer
- a floating point number
- part of a vertex
- or any structured data

Conclusion: All higher-level constructs (vectors, matrices, geometry) are built from:

$$\text{Bits} \to \text{Bytes} \to \text{Floats} \to \text{Mathematical Objects} \tag{19}$$

2 Geometry Representation in Memory

2.1 Vertices (Position Vectors)

A **vertex** represents a point in 3D space.

In its simplest form, a vertex consists of three floating-point values:

$$\mathbf{v} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \in \mathbb{R}^3 \tag{20}$$

For transformation purposes, vertices are often expressed in homogeneous coordinates:

$$\mathbf{v}_h = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \in \mathbb{R}^4 \tag{21}$$

Memory Representation: Each coordinate is a 32-bit float:

$$\text{vertex} = [x, y, z] \tag{22}$$

$$\text{size of vertex} = 3 \times 4 \text{ bytes} = 12 \text{ bytes} \tag{23}$$

Thus, a vertex buffer is:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_n] \tag{24}$$

2.2 Triangles (Faces)

The fundamental rendering primitive is the **triangle**.

A triangle is defined by three vertices:

$$\triangle = (\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2) \tag{25}$$

Geometrically, any point inside the triangle can be expressed using barycentric coordinates:

$$\mathbf{p} = \alpha\mathbf{v}_0 + \beta\mathbf{v}_1 + \gamma\mathbf{v}_2 \tag{26}$$

subject to:

$$\alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0 \tag{27}$$

Memory Representation (Non-Indexed): Each triangle stores three full vertices:

$$\text{triangle size} = 3 \times 12 \text{ bytes} = 36 \text{ bytes} \tag{28}$$

2.3 Indexed vs Non-Indexed Geometry

Non-Indexed Representation: Each triangle explicitly stores its vertices:

$$T = [\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2] \tag{29}$$

This leads to duplication of shared vertices.

Indexed Representation: Vertices are stored once in a vertex buffer:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_n] \tag{30}$$

Triangles are defined using indices:

$$F = [(i_0, i_1, i_2), (i_3, i_4, i_5), (i_6, i_7, i_8)] \tag{31}$$

Where winding order is used to specify face direction.
Each index refers to a vertex:

$$\triangle_k = (\mathbf{v}_{i_a}, \mathbf{v}_{i_b}, \mathbf{v}_{i_c}) \tag{32}$$

Memory Efficiency: Let:

$$n_v = \text{number of vertices}, \quad n_t = \text{number of triangles} \tag{33}$$

Then:

$$\text{non-indexed size} = 36 \cdot n_t \tag{34}$$

$$\text{indexed size} = 12 \cdot n_v + 12 \cdot n_t \tag{35}$$

(assuming 32-bit indices)

2.4 Meshes (Objects)

A **mesh** (object) consists of:

- A vertex buffer V
- An index buffer F

$$\mathcal{M} = (V, F) \tag{36}$$

The mesh is defined in **local (object) space**, meaning:

$$\mathbf{v}_{local} \in \mathbb{R}^3 \tag{37}$$

All vertices are expressed relative to the object's origin:

$$\mathbf{o}_{local} = (0, 0, 0) \tag{38}$$

2.5 Scenes as Collections of Objects

A **scene** consists of multiple objects:

$$\mathcal{S} = \{\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_k\} \tag{39}$$

Each object has an associated transformation:

$$M_i \in \mathbb{R}^{4\times 4} \tag{40}$$

which maps local coordinates to world coordinates:

$$\mathbf{v}_{world} = M_i \mathbf{v}_{local} \tag{41}$$

Global Interpretation: All objects share a common coordinate system:

$$\mathbf{v}_{world} \in \mathbb{R}^3 \tag{42}$$

Thus, the scene becomes a unified collection of geometry in world space.

Final Structure in Memory:

$$\text{Scene} = \big\{(V_1, F_1, M_1), (V_2, F_2, M_2), \ldots\big\} \tag{43}$$

Each component ultimately reduces to:

$$\text{Bytes} \to \text{Floats} \to \text{Vertices} \to \text{Triangles} \to \text{Meshes} \to \text{Scene} \tag{44}$$